

Relatório de **ALGAV Sprint 2**

Autores:

António Pereira –
1220754

Alejandro Vieira –
1211981

João Pinto – 1221294

1. Explicação do Código Prolog para Gestão de Cirurgias Hospitalares Fornecido

Definições Dinâmicas

Os predicados dinâmicos são definidos para que possam ser modificados em tempo de execução:

```
:- dynamic availability/3.  
:- dynamic agenda_staff/3.  
:- dynamic agenda_staff1/3.  
:-dynamic agenda_operation_room/3.  
:-dynamic agenda_operation_room1/3.  
:-dynamic better_sol/5.
```

Essas definições permitem a inserção, remoção e atualização dos dados durante a execução, essencial para modelar agendas dinâmicas.

Agendas e Horários

- **agenda_staff/3 e timetable/3:** Definem as agendas dos médicos (staff) e seus horários de trabalho no formato (HoraInício, HoraFim).
- **agenda_operation_room/3:** Define os horários de ocupação das salas de operação.

```
agenda_staff(d003,20241028,[ (720,790,m01), (910,980,m02) ] ).  
timetable(d001,20241028,(480,1200)) .
```

Recursos e Cirurgias

- **staff/4**: Lista os médicos, suas especialidades e as cirurgias que podem realizar.
- **surgery/4**: Define os tempos necessários para diferentes etapas de uma cirurgia (anestesia, operação e limpeza).
- **surgery_id/2**: Relaciona códigos únicos de cirurgia aos tipos de cirurgia.

```
staff(d003, doctor, orthopaedist, [so2, so3, so4]).
surgery(so2, 45, 60, 45).
```

Processos de Agendamento

1. Disponibilidade

Predicados como **free_agenda0/2** e **free_agenda1/2** calculam os intervalos livres na agenda de um recurso (médico ou sala), considerando horários já ocupados.

```
free_agenda0([], [(0, 1440)]).
free_agenda0([(0, Tfin, _) | LT], LT1) :- !, free_agenda1([(0, Tfin, _) | LT], LT1).
free_agenda0([(Tin, Tfin, _) | LT], [(0, T1) | LT1]) :- T1 is Tin-1,
    free_agenda1([(Tin, Tfin, _) | LT], LT1).

free_agenda1([(_, Tfin, _) | LT], [(T1, 1440)]):-Tfin\==1440, !, T1 is Tfin+1.
free_agenda1([(_, _, _) | LT], []).
free_agenda1([(_, T, _) | LT], [(T1, Tfin2, _) | LT], LT1) :- Tx is T+1, T1==Tx, !,
    free_agenda1([(T1, Tfin2, _) | LT], LT1).
free_agenda1([(_, Tfin1, _) | LT], [(Tin2, Tfin2, _) | LT], [(T1, T2) | LT1]) :- T1 is Tfin1+1, T2 is Tin2-1,
    free_agenda1([(Tin2, Tfin2, _) | LT], LT1).
```

2. Adaptação de Horários

O predicado **adapt_timetable/4** ajusta os intervalos disponíveis com base nos horários de trabalho do médico ou sala

```
adapt_timetable(D, Date, LFA, LFA2) :- timetable(D, Date, (InTime, FinTime)), treatin(InTime, LFA, LFA1), treatfin(FinTime, LFA1, LFA2).
```

3. Interseção de Agendas

intersect_all_agendas/3 calcula os horários compatíveis entre várias agendas (e.g., médicos designados para uma mesma cirurgia).

```
intersect_all_agendas([Name], Date, LA) :- !, availability(Name, Date, LA).  
intersect_all_agendas([Name | LNames], Date, LI) :-  
    availability(Name, Date, LA),  
    intersect_all_agendas(LNames, Date, LI1),  
    intersect_2_agendas(LA, LI1, LI).
```

4. Remoção de Conflitos

remove_unf_intervals/3 elimina intervalos inadequados, garantindo que o tempo disponível seja suficiente para a cirurgia.

```
remove_unf_intervals(_, [], []).  
remove_unf_intervals(TSurgery, [(Tin, Tfin) | LA], [(Tin, Tfin) | LA1]) :- DT is Tfin - Tin + 1,  
    TSurgery =<DT, !,  
        remove_unf_intervals(TSurgery, LA, LA1).  
remove_unf_intervals(TSurgery, [_ | LA], LA1) :- remove_unf_intervals(TSurgery, LA, LA1).
```

5. Inserção na Agenda

O predicado **insert_agenda/3** insere uma nova cirurgia na agenda de médicos ou salas.

```
insert_agenda((TinS,TfinS,OpCode),[],[(TinS,TfinS,OpCode)]).
insert_agenda((TinS,TfinS,OpCode),[(Tin,Tfin,OpCode1)|LA],[(TinS,TfinS,OpCode),(Tin,Tfin,OpCode1)|LA]):-TfinS<Tin,!
insert_agenda((TinS,TfinS,OpCode),[(Tin,Tfin,OpCode1)|LA],[(Tin,Tfin,OpCode1)|LA1]):-insert_agenda((TinS,TfinS,OpCode),LA,LA1).

insert_agenda_doctors(_,_,[]).
insert_agenda_doctors((TinS,TfinS,OpCode),Day,[Doctor|LDoctors]):-
    retract(agenda_staff1(Doctor,Day,Agenda)),
    insert_agenda((TinS,TfinS,OpCode),Agenda,Agenda1),
    assert(agenda_staff1(Doctor,Day,Agenda1)),
    insert_agenda_doctors((TinS,TfinS,OpCode),Day,LDoctors).
```

Agendamento Globa

- **schedule_all_surgries/3**: Automatiza o agendamento de todas as cirurgias para um dia e sala específicos, verificando compatibilidade entre recursos e disponibilidades.
- **availability_all_surgeries/3**: Garante que todas as cirurgias sejam alocadas sem conflitos.

```
schedule_all_surgeries(Room,Day):-
    retractall(agenda_staff1(_,_,_)),
    retractall(agenda_operation_room1(_,_,_)),
    retractall(availability(_,_,_)),
    findall(_,(agenda_staff(D,Day,Agenda),assertz(agenda_staff1(D,Day,Agenda))),_),
    agenda_operation_room(Or,Date,Agenda),assert(agenda_operation_room1(Or,Date,Agenda)),
    findall(_,(agenda_staff1(D,Date,L),free_agenda0(L,LFA),adapt_timetable(D,Date,LFA,LFA2),assertz(availability(D,Date,LFA2))),_),
    findall(OpCode,surgery_id(OpCode,_),LOpCode),

    availability_all_surgeries(LOpCode,Room,Day),!.

availability_all_surgeries([],_,_).
availability_all_surgeries([OpCode|LOpCode],Room,Day):-
    surgery_id(OpCode,OpType),surgery(OpType,_,TSurgery,_),
    availability_operation(OpCode,Room,Day,LPossibilities,LDoctors),
    schedule_first_interval(TSurgery,LPossibilities,(TinS,TfinS)),
    retract(agenda_operation_room1(Room,Day,Agenda)),
    insert_agenda((TinS,TfinS,OpCode),Agenda,Agenda1),
    assertz(agenda_operation_room1(Room,Day,Agenda1)),
    insert_agenda_doctors((TinS,TfinS,OpCode),Day,LDoctors),
    availability_all_surgeries(LOpCode,Room,Day).
```

Otimização

- **obtain_better_sol/4**: Gera a melhor solução possível para o agendamento, minimizando o tempo total das operações.
 - Usa permutações de cirurgias para encontrar a alocação ideal.
 - Avalia cada solução com base no tempo final da última operação.

```

obtain_better_sol(Room,Day,AgOpRoomBetter,LAgDoctorsBetter,TFinOp):-
    get_time(Ti),
    (obtain_better_sol1(Room,Day);true),
    retract(better_sol(Day,Room,AgOpRoomBetter,LAgDoctorsBetter,TFinOp
)),
    write('Final Result: AgOpRoomBetter='),write(AgOpRoomBetter),nl,
    write('LAgDoctorsBetter='),write(LAgDoctorsBetter),nl,
    write('TFinOp='),write(TFinOp),nl,
    get_time(Tf),
    T is Tf-Ti,
    write('Tempo de geracao da solucao:'),write(T),nl.

```

Funcionamento Geral

1. Carrega as agendas iniciais dos médicos e salas.
2. Calcula horários livres, adaptando-os conforme necessário.
3. Realiza o agendamento das cirurgias, considerando compatibilidade e tempos mínimos necessários.
4. Retorna uma solução otimizada.

Objetivo do Código

Este sistema é essencial para gerir eficientemente os recursos hospitalares, garantindo o uso otimizado de salas de operação e o cumprimento dos horários de trabalho dos profissionais. É flexível e pode ser adaptado a diferentes cenários hospitalares.

2. Análise de Complexidade

Para avaliar a complexidade do algoritmo de agendamento, incrementamos gradualmente o número de cirurgias a serem alocadas, medindo o impacto no tempo de execução e no número de soluções geradas. O algoritmo baseia-se na análise de todas as permutações possíveis das cirurgias, resultando em uma escala de crescimento **fatorial**. Essa característica implica que, conforme o número de cirurgias aumenta, o espaço de busca cresce exponencialmente, tornando-se rapidamente inviável para capacidades computacionais padrão.

Comando Utilizado nos Testes

Os testes foram realizados utilizando o comando:

```
?- obtain_better_sol(or1,20241028,_,_,_).
```

Esse comando gerou as soluções possíveis para um determinado número de cirurgias e calculou o tempo necessário para encontrar a solução ótima. A seguir, apresentamos um exemplo do que foi obtido na consola:

Exemplo de saída:

```
Analysing for LOpCode=[sol00001,sol00002,sol00003]
now: FinTime1=865 Agenda=[(520,579,sol00000),(580,639,sol00001),(640,729,sol00002),(791,865,sol00003),(1000,1059,sol099999)]

best solution updated
Analysing for LOpCode=[sol00001,sol00003,sol00002]
now: FinTime1=804 Agenda=[(520,579,sol00000),(580,639,sol00001),(640,714,sol00003),(715,804,sol00002),(1000,1059,sol099999)]

best solution updated
Analysing for LOpCode=[sol00002,sol00001,sol00003]
now: FinTime1=1134 Agenda=[(520,579,sol00000),(580,669,sol00002),(791,850,sol00001),(1000,1059,sol099999),(1060,1134,sol00003)]
Analysing for LOpCode=[sol00002,sol00003,sol00001]
now: FinTime1=925 Agenda=[(520,579,sol00000),(580,669,sol00002),(791,865,sol00003),(866,925,sol00001),(1000,1059,sol099999)]

Analysing for LOpCode=[sol00003,sol00001,sol00002]
now: FinTime1=804 Agenda=[(520,579,sol00000),(580,654,sol00003),(655,714,sol00001),(715,804,sol00002),(1000,1059,sol099999)]

Analysing for LOpCode=[sol00003,sol00002,sol00001]
now: FinTime1=850 Agenda=[(520,579,sol00000),(580,654,sol00003),(655,744,sol00002),(791,850,sol00001),(1000,1059,sol099999)]

Final Result: AgOpRoomBetter=[(520,579,sol00000),(580,639,sol00001),(640,714,sol00003),(715,804,sol00002),(1000,1059,sol099999)]
LAgDoctorsBetter=[(d001,[(580,639,sol00001),(720,790,m01),(1080,1140,c01)]),(d002,[(715,804,sol00002),(850,900,m02),(901,960,m02),(1380,1440,c02)]),(d003,[(640,714,sol00003),(720,790,m01),(910,980,m02)])]
TFinOp=804
Tempo de geracao da solucao:0.11356711387634277
true.
```

Tabela de Resultados Obtidos

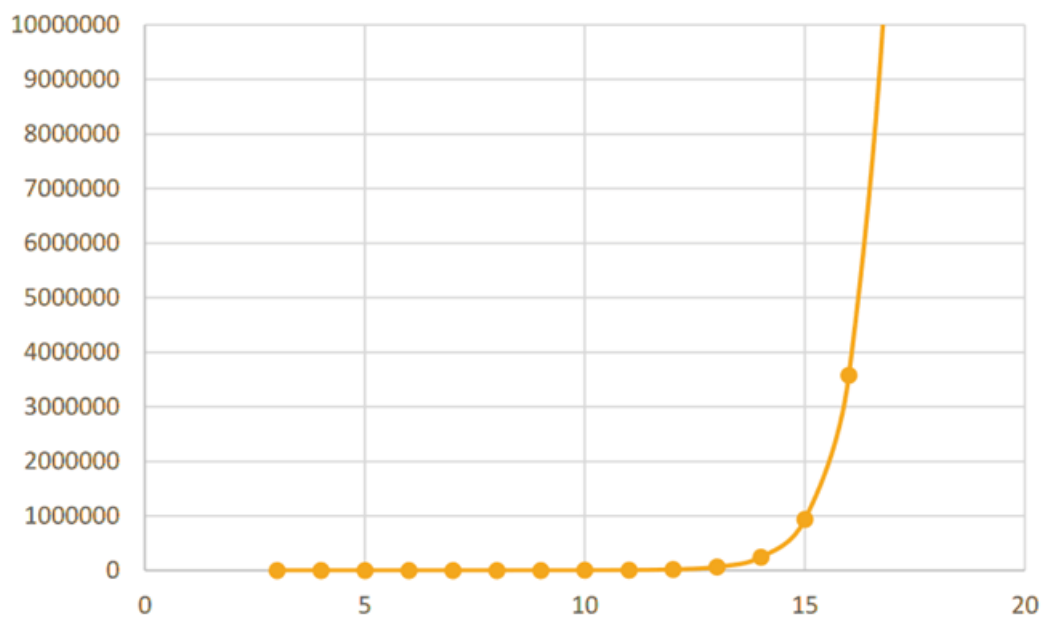
Os resultados práticos dos testes são apresentados na tabela abaixo. Cada linha mostra a relação entre o número de cirurgias a serem agendadas, o número total de soluções possíveis (permutações), a duração final da última cirurgia e o tempo necessário para a geração da solução ótima.

Nº de Cirurgias	Nº de Soluções	Agenda Gerada (Resumo)	Final time (min)	Generate time (s)
3	6	AgOpRoomBetter=[(520,579,so100000),(580,639,so100001),(640,714,so100003),(715,804,so100002),(1000,1059,so0999999)]	804	0.1136
4	24	AgOpRoomBetter=[(520,579,so100000),(580,654,so100003),(655,714,so100004),(715,804,so100002),(805,864,so100001),(1000,1059,so0999999)]	864	0.1387
5	120	AgOpRoomBetter=[(520,579,so100000),(580,639,so100004),(640,714,so100005),(715,804,so100002),(805,879,so100003),(880,939,so100001),(1000,1059,so0999999)]	939	1.9919
6	720	AgOpRoomBetter=[(520,579,so100000),(580,639,so100004),(640,714,so100005),(715,804,so100002),(805,879,so100003),(880,939,so100001),(940,999,so100006),(1000,1059,so0999999)]	999	5.4084
7	5040	AgOpRoomBetter=[(520,579,so100000),(580,639,so100004),(640,714,so100005),(715,804,so100002),(805,879,so100003),(880,939,so100001),(940,999,so100006),(1000,1059,so0999999),(1060,1149,so100007)]	1149	16.6232
8	40320	AgOpRoomBetter=[(520,579,so100000),(580,639,so100004),(640,699,so100008),(700,789,so100002),(791,865,so100003),(866,925,so100001),(926,985,so100006),(1000,1059,so0999999),(1060,1134,so100005),(1135,1224,so100007)]	1224	30.3059
9	36288	AgOpRoomBetter=[(520,579,so100000),(580,639,so100004),(640,699,so100008),(700,789,so100002),(790,849,so100009),(850,909,so100001),(910,969,so100006),(1000,1059,so0999999),(1060,1134,so100005),(1135,1209,so100003),(1210,1299,so100007)]	1299	71.1453
10	36288	AgOpRoomBetter=[(520,579,so100000),(580,639,so100004),(640,699,so100008),(700,759,so100009),(791,865,so100003),(866,925,so100001),(926,985,so100006),(1000,1059,so0999999),(1060,1134,so100005),(1135,1224,so100007),(1225,1284,so100010),(1285,1374,so100002)]	1374	3573.8958
11	39916800	n/a	n/a	n/a
12	479001600	n/a	n/a	n/a
13	6227020800	n/a	n/a	n/a

Discussão dos Resultados

Os resultados confirmam a natureza fatorial do algoritmo. Enquanto para um pequeno número de cirurgias (3 ou 4) o tempo de execução é aceitável, a partir de 6 cirurgias o crescimento exponencial torna o tempo de execução proibitivo para aplicações práticas em sistemas hospitalares de alta demanda.

A complexidade algorítmica é um gargalo que exige atenção para que o sistema seja viável em larga escala. Embora o método atual seja suficiente para experimentos e validação, otimizações como heurísticas ou técnicas de paralelização são fundamentais para lidar com cenários maiores.



3. Heurísticas e comparação com solução ótima:

Primeira Heurística

A primeira heurística busca otimizar o agendamento das cirurgias, minimizando o tempo ocioso dos recursos e garantindo que as cirurgias sejam realizadas no melhor horário possível. O processo de agendamento é realizado de forma iterativa, onde cada cirurgia é alocada considerando as restrições de tempo dos médicos, das salas e a duração da cirurgia.

A principal heurística aplicada no algoritmo é a organização das cirurgias pelo tempo de execução. O predicado **heuristic_by_time/4** é responsável por ordenar as cirurgias pela sua duração, com as cirurgias de menor duração sendo agendadas primeiro. Esse comportamento visa a melhoria da utilização dos recursos, pois as cirurgias mais curtas ocupam menos tempo das salas de operação e dos médicos, permitindo o agendamento de mais procedimentos ao longo do dia.

```
heuristic_by_time([], _, _, []).
heuristic_by_time([OpCode | LOpCode], Room, Day, SortedLOpCode) :-
    surgery_id(OpCode, OpType),
    surgery(OpType, _, TSurgery, _),
    availability_operation(OpCode, Room, Day, LPossibilities, _),
    (LPossibilities \= [] ->
        schedule_first_interval(TSurgery, LPossibilities, (TinS, _)),
        heuristic_by_time(LOpCode, Room, Day, PartialSorted),
        append([(OpCode, TinS)], PartialSorted, PartialSorted1),
        inverter(PartialSorted1, NewPartialSorted),
        sort_op_list(NewPartialSorted, SortedList),
        SortedLOpCode = SortedList;
    SortedLOpCode = []
    ).
```

Além disso, o algoritmo considera as interações entre as disponibilidades dos médicos e das salas de operação para garantir que as cirurgias sejam alocadas de forma otimizada. Quando uma cirurgia é marcada, as disponibilidades dos recursos são ajustadas dinamicamente, recalculando os horários livres para os próximos agendamentos. Isso é realizado pelo predicado **availability_operation/4**, que tem um papel fundamental na determinação da disponibilidade combinada entre as agendas dos médicos e das salas de operação.

```
availability_operation(OpCode,Room,Day,LPossibilities,LDoctors):-surgery_id(OpCode
,OpType),surgery(OpType,_,TSurgery,_),
    findall(Doctor,assignment_surgery(OpCode,Doctor),LDoctors),
    intersect_all_agendas(LDoctors,Day,LA),
    agenda_operation_room1(Room,Day,LAagenda),
    free_agenda0(LAagenda,LFAgRoom),
    intersect_2_agendas(LA,LFAgRoom,LIntAgDoctorsRoom),
    remove_unf_intervals(TSurgery,LIntAgDoctorsRoom,LPossibilities).
```

Este predicado verifica se uma cirurgia pode ser alocada em uma sala de operação, levando em conta tanto a disponibilidade do médico quanto a agenda da sala de operação.

Resultado gerado:

```
Sala de Operações orl
Agenda para sala orl no dia 20241028: [(480,539,so100006),(540,614,so100011),(615,674,so100010),(675,76
4,so100009),(765,824,so100013),(825,884,so100012),(885,944,so100001),(981,1055,so100003)]

Agendas dos Médicos:
Médico d002: [(675,764,so100009),(850,900,m02),(901,960,m02),(1380,1440,c02)]
Médico d004: [(675,764,so100009),(765,824,so100013)]
Médico d001: [(480,539,so100006),(540,614,so100011),(720,790,m01),(825,884,so100012),(885,944,so100001)
,(1080,1140,c01)]
Médico d003: [(615,674,so100010),(720,790,m01),(910,980,m02),(981,1055,so100003)]

Tempo de geracao da solucao:0.009904861450195312
LRoom = [(480, 539, so100006), (540, 614, so100011), (615, 674, so100010), (675, 764, so100009), (765, 82
4, so100013), (825, 884, so100012), (885, 944, so100001), (981, ..., ...)].
LDAgendas = [(d001, [(480, 539, so100006), (540, 614, so100011), (720, 790, m01), (825, 884, so100012), (
885, 944, so100001), (1080, ..., ...)]), (d002, [(675, 764, so100009), (850, 900, m02), (901, 960, m02),
(1380, 1440, c02)]), (d003, [(615, 674, so100010), (720, 790, m01), (910, 980, m02), (981, ..., ...)]), (
d001, [(480, 539, so100006), (540, 614, so100011), (720, ..., ...), (... , ...) |...]), (d002, [(675, 764,
so100009), (850, ..., ...), (... , ...) |...]), (d002, [(675, ..., ...), (... , ...) |...]), (d003, [(..., ...
) |...]), (d001, [... |...]), (... , ...) |...].
```

Segunda Heurística

A heurística desenvolvida visa otimizar o agendamento das cirurgias, minimizando o tempo ocioso dos médicos e das salas de operação, enquanto maximiza a utilização dos recursos disponíveis. O processo de agendamento é realizado de forma iterativa, onde, a cada iteração, uma cirurgia é alocada com base na disponibilidade dos médicos e das salas, levando em consideração a ocupação de cada médico e a duração das cirurgias.

O algoritmo começa com o cálculo do tempo livre dos médicos através da função **calculate_free_time/2**, que soma os intervalos de tempo livre de cada médico, considerando os seus horários de trabalho. A função **get_total_time_free/3** integra a obtenção das agendas dos médicos e calcula o tempo livre total, para então determinar a porcentagem de ocupação de cada médico usando o predicado **staff_occupation_percentage/3**

```
calculate_free_time([], 0).
calculate_free_time([(In, Fin)|LT], TotalFreeTime) :-
    FreeTime is Fin - In,
    calculate_free_time(LT, RemainingTime),
    TotalFreeTime is RemainingTime + FreeTime.

get_total_time_free(D, Date, TotalFreeTime) :-
    agenda_staff(D, Date, L),
    free_agenda0(L, LFA),
    adapt_timetable(D, Date, LFA, LFA2),
    calculate_free_time(LFA2, TotalFreeTime).
```

O predicativo **staff_occupation_percentage/3** calcula a porcentagem da ocupação de cada médico com base no tempo livre disponível em

relação ao horário total de trabalho. Esta função é utilizada pela função **all_doctor_occupation/2**, que gera uma lista contendo a ocupação de todos os médicos no dia especificado.

```
staff_occupation_percentage(D,Date,R):-
    get_total_time_free(D,Date,TFT),
    calculate_working_hours(D,Date,WH),
    R is (TFT/WH) * 100.

all_doctor_occupation(Date,[(Doctor, OccupationPercent) | R]) :-
    findall((Doctor, OccupationPercent), (staff_occupation_percentage(Doctor, Date
, OccupationPercent))), R).
```

Este cálculo é crucial para priorizar os médicos mais ocupados, que são selecionados pela função **sort_doctors_by_occupation/2**, ordenando-os em ordem decrescente de ocupação.

```
sort_doctors_by_occupation(DoctorOccupations, SortedDoctors) :-
    sort(2, @>=, DoctorOccupations, SortedDoctorsWithOccupation),
    findall(Doctor, member((Doctor, _), SortedDoctorsWithOccupation), SortedDoctors).
```

Uma vez que os médicos estão ordenados pela sua ocupação, o algoritmo segue para a alocação das cirurgias. A função **search_surgery_for_doctor/3** verifica quais cirurgias estão associadas a cada médico e tenta alocar a cirurgia que se ajusta à sua agenda.

```
search_surgery_for_doctor([], _, []) :-
    !.
search_surgery_for_doctor(_, [], []) :-
    !.
search_surgery_for_doctor(LOpCode, [Doctor | Rest], OpCode) :-
    findall(OpCode, (member(OpCode, LOpCode), assignment_surgery(OpCode, Doctor)),
    LOpCodeForDoctor),

    (
        LOpCodeForDoctor \= []
    -> select(OpCode, LOpCodeForDoctor, _),
        true
    ;
        ▲ search surgery for doctor(LOpCode, Rest, OpCode)
    ).

select_surgery([OpCode | ], OpCode).
```


Para garantir que as cirurgias sejam agendadas corretamente, a função **mark_surgery_if_possible/3** verifica a compatibilidade dos horários dos médicos com os da sala de operação. Ela faz isso ao analisar a disponibilidade da sala e dos médicos para o dia específico e procura por horários livres onde a cirurgia pode ser realizada sem conflito.

```
mark_surgery_if_possible(OpCode,Room,Date) :-
    surgery_id(OpCode,OpType),surgery(OpType,_,TSurgery,_),
    findall(Doctor,assignment_surgery(OpCode,Doctor),LDoctors),
    intersect_all_agendas(LDoctors, Date, LA),
    agenda_operation_room1(Room, Date, LAgenda),
    free_agenda0(LAgenda, LFAgRoom),
    intersect_2_agendas(LA, LFAgRoom, LIntAgDoctorsRoom),
    remove_unf_intervals(TSurgery, LIntAgDoctorsRoom, LPossibilities),
    (
        LPossibilities \= []
    ->
        schedule_first_interval(TSurgery,LPossibilities,(TinS,TfinS)),
        retract(agenda_operation_room1(Room,Day,Agenda)),
        insert_agenda((TinS,TfinS,OpCode),Agenda,Agenda1),
        assertz(agenda_operation_room1(Room,Day,Agenda1)),
        insert_agenda_doctors((TinS,TfinS,OpCode),Day,LDoctors),
        write('Cirurgia marcada com sucesso!')
    ;
        write('Não há horários disponíveis para marcar a cirurgia.'), nl
    ).
```

A alocação de cirurgias é realizada de forma iterativa pela função **attempt_to_mark_surgeries/3**, que tenta agendar cada cirurgia para o médico mais ocupado, começando pelos médicos com maior taxa de ocupação. A função continua até que todas as cirurgias sejam alocadas ou até que não seja mais possível encontrar horários adequados. O processo é eficiente ao garantir que os recursos sejam utilizados de forma otimizada, minimizando o tempo ocioso e maximizando a utilização das salas e médicos disponíveis.

```

attempt_to_mark_surgeries([],_,_) :-
    write('Todas as cirurgias foram marcadas com sucesso!').
attempt_to_mark_surgeries(LOpCode,Room,Day) :-
    all_doctor_occupation(Day, DoctorOccupations),
    find_most_occupied_doctor(DoctorOccupations, SortedDoctors),
    search_surgery_for_doctor(LOpCode,SortedDoctors, OpCode),
    mark_surgery_if_possible(OpCode,Room,Day),
    remove_surgery_from_list(OpCode,LOpCode,NewLOpCode),
    attempt_to_mark_surgeries(NewLOpCode,Room,Day).

```

Finalmente, a função **schedule_all_surgeries/2** executa todo o processo de agendamento, começando pela preparação das agendas e culminando com a marcação das cirurgias, enquanto apresenta o resultado final com as agendas dos médicos e da sala de operação, além do tempo total de execução da solução.

```

schedule_all_surgeries(Room,Day) :-
    get_time(Ti),
    retractall(agenda_staff1(_,_,_)),
    retractall(agenda_operation_room1(_,_,_)),
    retractall(availability(_,_,_)),
    findall(_, (agenda_staff(D,Day,Agenda), assertz(agenda_staff1(D,Day,Agenda))), _)
,
    agenda_operation_room(Or,Date,Agenda), assert(agenda_operation_room1(Or,Date,Agenda)),
    findall(_, (agenda_staff1(D,Date,L), free_agenda0(L,LFA), adapt_timetable(D,Date,LFA,LFA2), assertz(availability(D,Date,LFA2))), _),
    findall(OpCode, surgery_id(OpCode,_), LOpCode),
    attempt_to_mark_surgeries(LOpCode,Room,Day), !,

%Exportação dos resultados para consola.
write("Resultados de agendamento para o dia ", write(Day), write(":\n\n"),
format("Sala de Operações ~w~n", [Room]),
( agenda_operation_room1(Room, Day, AgendaRoom),
  AgendaRoom \= [] ->
    format(" Agenda para sala ~w no dia ~w: ~w~n", [Room, Day, AgendaRoom])
; write(" Nenhuma cirurgia agendada.\n")
),
write("\nAgendas dos Médicos:\n"),
findall(Doctor, agenda_staff1(Doctor, Day, _), Doctors),
forall(member(Doctor, Doctors),
    (agenda_staff1(Doctor, Day, AgendaDoc),
     format(" Médico ~w: ~w~n", [Doctor, AgendaDoc]))
),
get_time(Tf),
T is Tf-Ti,
write('Tempo de geracao da solucao:'),write(T),nl.

```

Resultado gerado:

Comparação das heurísticas com a melhor solução

Nº	Best solution	Final time (min)	Generate time (s)	Heuristic one	Final time	Generate time	Heuristic two	Final time	Generate time
3	[(520,579,so100000),(580,639,so100001),(640,714,so100003),(715,804,so100002),(1000,1059,so099999)]	804	0.1136	[(480,539,so100001),(540,614,so100003),(615,704,so100002)]	704	0.0081	[(520,594,so100003),(595,684,so100002),(791,850,so100001)]	850	0.0058
4	[(520,579,so100000),(580,654,so100003),(655,714,so100004),(715,804,so100002),(805,864,so100001),(1000,1059,so099999)]	864	0.1387	[(480,539,so100001),(540,599,so100004),(600,689,so100002),(791,865,so100003)]	865	0.0077	[(520,594,so100003),(595,684,so100002),(791,850,so100001),(961,1020,so100004)]	1020	0.0052
5	[(520,579,so100000),(580,639,so100004),(640,714,so100005),(715,804,so100002),(805,879,so100003),(880,939,so100001),(1000,1059,so099999)]	939	1.9919	[(480,539,so100001),(540,599,so100004),(600,674,so100005),(675,764,so100002),(791,865,so100003)]	865	0.0074	[(520,594,so100003),(595,669,so100005),(670,759,so100002),(791,850,so100001),(961,1020,so100004)]	1020	0.0052
6	[(520,579,so100000),(580,639,so100004),(640,714,so100005),(715,804,so100002),(805,879,so100003),(880,939,so100001),(940,999,so100006),(1000,1059,so099999)]	999	5.4084	[(480,539,so100006),(540,629,so100002),(630,704,so100003),(791,850,so100001),(961,1020,so100004),(1021,1095,so100005)]	1095	0.0076	[(520,594,so100003),(595,669,so100005),(670,759,so100002),(791,850,so100001),(851,910,so100006),(961,1020,so100004)]	1020	0.0052
7	[(520,579,so100000),(580,639,so100004),(640,714,so100005),(715,804,so100002),(805,879,so100003),(880,939,so100001),(940,999,so100006),(1000,1059,so099999),(1060,1149,so100007)]	1149	16.6232	[(480,539,so100006),(540,629,so100002),(630,704,so100003),(791,850,so100001),(961,1020,so100004),(1021,1110,so100007),(1111,1185,so100005)]	1185	0.0080	[(520,594,so100003),(595,669,so100005),(670,759,so100002),(791,880,so100007),(881,940,so100001),(961,1020,so100004),(1141,1200,so100006)]	1200	0.0058

9	[(520,579,so100000),(580,639,so100004), (640,699,so100008),(700,789,so100002), (790,849,so100009),(850,909,so100001), (910,969,so100006),(1000,1059,so099999), (1060,1134,so100005),(1135,1209,so100003), (1210,1299,so100007)]	1299	71.1453	[(480,539,so100006),(540,629,so100002), (630,704,so100003),(705,794,so100009), (795,884,so100007)]	884	0.0130	[(520,594,so100003),(620,679,so100008), (680,739,so100009),(791,880,so100007), (881,940,so100001),(981,1055,so100005), (1056,1145,so100002)]	1145	0.0074
10	[(520,579,so100000),(580,639,so100004), (640,699,so100008),(700,759,so100009), (791,865,so100003),(866,925,so100001), (926,985,so100006),(1000,1059,so099999), (1060,1134,so100005),(1135,1224,so100007), (1225,1284,so100010),(1285,1374,so100002)]	1374	3573.8958	[(480,539,so100006),(540,629,so100002), (630,704,so100005),(705,794,so100009), (795,854,so100008),(855,914,so100001), (961,1020,so100004),(1021,1080,so100010), (1081,1170,so100007),(1171,1245,so100003)]	1245	0.0133	[(520,594,so100003),(620,679,so100008), (680,739,so100009),(791,880,so100007), (881,940,so100001),(981,1055,so100005), (1056,1115,so100010), (1116,1205,so100002)]	1145	0.0074
11	n/a	n/a	n/a	[(480,539,so100006),(540,599,so100001), (600,689,so100007),(690,779,so100002)]	779	0.0134	[(520,594,so100003),(620,679,so100008), (680,739,so100009),(791,880,so100007), (881,940,so100001),(981,1055,so100005), (1056,1115,so100010), (1116,1205,so100002)]	1205	0.0078
12	n/a	n/a	n/a	[(480,539,so100006),(540,614,so100011), (615,674,so100010),(675,764,so100009), (791,850,so100001),(851,910,so100012), (981,1070,so100007),(1071,1145,so100003)]	1145	0.0121	[(520,594,so100003),(620,679,so100008), (680,739,so100009),(791,880,so100007), (881,940,so100001),(981,1055,so100005), (1056,1115,so100010), (1116,1205,so100002)]	1205	0.0073
13	n/a	n/a	n/a	[(480,539,so100006),(540,614,so100011), (615,674,so100010),(675,764,so100009), (765,824,so100013),(825,884,so100012), (885,944,so100001),(981,1055,so100003)]	1055	0.0139	[(520,594,so100003),(620,679,so100008), (680,739,so100009),(740,799,so100013), (800,889,so100007),(890,949,so100001), (981,1055,so100005), (1056,1115,so100010), (1116,1205,so100002)]	1205	0.0083

4. Adaptação dos métodos para incluir todo o staff e fases de operação:

Para adaptar o algoritmo desenvolvido a um contexto do mundo real, foram feitas modificações para permitir a inclusão de diversos membros da equipe e diferentes etapas de operação

Primeira Heurística

O processo de integrar toda a equipe exige algumas adaptações no algoritmo. Foram adicionados os seguintes elementos como exemplos para ilustrar as mudanças necessárias.

```
agenda_staff(d001,20241028,[(720,790,m01),(1080,1140,c01)]).
agenda_staff(d002,20241028,[(901,960,m02),(1380,1440,c02)]).
agenda_staff(d003,20241028,[(720,790,m01),(910,980,m02)]).
agenda_staff(d004,20241028,[]).
agenda_staff(d005,20241028,[]).
agenda_staff(n001,20241028,[]).
agenda_staff(n002,20241028,[]).
agenda_staff(n003,20241028,[]).
agenda_staff(a001,20241028,[]).
agenda_staff(n004,20241028,[]).
agenda_staff(n005,20241028,[]).
agenda_staff(n006,20241028,[]).
agenda_staff(a002,20241028,[]).
```

```
timetable(d001,20241028,(480,1200)).
timetable(d002,20241028,(500,1440)).
timetable(d003,20241028,(520,1320)).
timetable(d004,20241028,(480,1200)).
timetable(d005,20241028,(500,1440)).
timetable(n001,20241028,(520,1320)).
timetable(n002,20241028,(480,1200)).
timetable(n003,20241028,(500,1440)).
timetable(a001,20241028,(520,1320)).
timetable(n004,20241028,(480,1200)).
timetable(n005,20241028,(500,1440)).
timetable(n006,20241028,(520,1320)).
timetable(a002,20241028,(520,1320)).
```

```

staff(d001,doctor,orthopaedist,[so2,so3,so4]).
staff(d002,doctor,orthopaedist,[so2,so3,so4]).
staff(d003,doctor,orthopaedist,[so2,so3,so4]).
staff(d004,doctor,anaesthetist,[so2,so3,so4]).
staff(d005,doctor,anaesthetist,[so2,so3,so4]).
staff(n001,nurse,instrumenting,[so2,so3,so4]).
staff(n002,nurse,cleaing,[so2,so3,so4]).
staff(n003,nurse,anaesthetist,[so2,so3,so4]).
staff(a001,technician,assistant,[so2,so3,so4]).
staff(n004,nurse,instrumenting,[so2,so3,so4]).
staff(n005,nurse,circulating,[so2,so3,so4]).
staff(n006,nurse,anaesthetist,[so2,so3,so4]).
staff(a002,technician,assistant,[so2,so3,so4]).

```

```

assignment_surgery(so100001,d001).
assignment_surgery(so100001,d004).
assignment_surgery(so100002,d002).
assignment_surgery(so100002,d005).
assignment_surgery(so100003,d003).
assignment_surgery(so100003,d004).
assignment_surgery(so100004,d001).
assignment_surgery(so100004,d005).
assignment_surgery(so100004,d002).
assignment_surgery(so100005,d002).
assignment_surgery(so100005,d003).
assignment_surgery(so100005,d005).

```

Para determinar, dentro de um grupo de profissionais disponíveis, quais serão selecionados para cada cirurgia, desenvolvemos um conjunto de predicados. Esses predicados permitem avaliar a ocupação do staff e organizá-los de acordo com uma ordem pré-definida, garantindo uma alocação eficiente e alinhada com as necessidades do procedimento.

```

staff_occupation_percentage(D,Date,R):-
    get_total_time_free(D,Date,TFT),
    calculate_working_hours(D,Date,WH),
    R is (TFT/WH) * 100.

all_staff_occupation(_, [], []).
all_staff_occupation(Date, [Staff | RestStaff], [(Staff, OccupationPercent) | RestResult]) :-
    staff_occupation_percentage(Staff, Date, OccupationPercent),
    all_staff_occupation(Date, RestStaff, RestResult).

```


Para realizar esse tipo de ordenação, é fundamental obter uma lista completa do staff, permitindo avaliar a ocupação de cada membro de forma precisa.

```
obtain_staff_speciality(Type, Specialty, LStaffNoDuplicates) :-  
    findall(StaffId, (  
        staff(StaffId, Type, Specialty, _)  
    ), LStaff),  
    remove_equals(LStaff, LStaffNoDuplicates).
```

Por meio do predicado **availability_all_surgeries/2**, é realizada a introdução de cada membro do staff e das respectivas fases das cirurgias, permitindo mapear a disponibilidade de forma integrada.

```
surgery_id(OpCode, OpType),  
surgery(OpType, TAnaes, TSurgery, TClean),  
total_time(TAnaes, TSurgery, TATotal),  
total_time(TATotal, TClean, TTotal),  
  
obtain_staff_speciality(doctor, anaesthetist, LADoctors),  
all_staff_occupation(Day, LADoctors, Result1),  
sort_staff_by_occupation(Result1, SortedStaff1),  
  
obtain_staff_speciality(nurse, anaesthetist, LANurses),  
all_staff_occupation(Day, LANurses, Result2),  
sort_staff_by_occupation(Result2, SortedStaff2),  
  
obtain_staff_speciality(technician, assistant, LMAAs),  
all_staff_occupation(Day, LMAAs, Result3),  
sort_staff_by_occupation(Result3, SortedStaff3),  
  
obtain_staff_speciality(nurse, instrumenting, LINurses),  
all_staff_occupation(Day, LINurses, Result4),  
sort_staff_by_occupation(Result4, SortedStaff4),  
  
obtain_staff_speciality(nurse, circulating, LCNurses),  
all_staff_occupation(Day, LCNurses, Result5),  
sort_staff_by_occupation(Result5, SortedStaff5),
```

Realiza-se uma análise detalhada das ocupações de cada membro do staff para determinar quais serão selecionados para a próxima cirurgia a ser agendada.

```
append([[AD],[AN],[IN],[CN],LDoctors,[MAA]],TotalStaff),
intersect all agendas(TotalStaff,Day,ATotalStaff),
```

As agendas de todos os membros do staff selecionado, incluindo os médicos, são cruzadas para identificar um horário comum disponível. O objetivo é garantir a compatibilidade dos horários e facilitar o agendamento da cirurgia.

```
intersect_2_agendas([],_,[]).
intersect_2_agendas([D|LD],[LA,LIT):- intersect_availability(D,LA,LI,LA1),
intersect_2_agendas(LD,LA1,LID),
append(LI,LID,LIT).
```

Por fim, foi adicionado um predicado responsável por retornar uma lista contendo todos os membros da equipe (staff), permitindo uma visualização completa dos envolvidos nas operações.

```
collect_staff_agendas(Day, LDAgendas) :-
findall(Staff, agenda_staff1(Staff, Day, _), AllStaff),
remove_equals(AllStaff, UniqueStaff),
findall((Staff, Agenda),
(member(Staff, UniqueStaff), agenda_staff1(Staff, Day, Agenda)),
LDAgendas
).
```

Resultado gerado:

Resultados de agendamento para o dia 20241028:

Sala de Operações orl

Agenda para sala orl no dia 20241028: [(520,699,so100002),(791,940,so100001),(981,1145,so100005)]

Agendas dos Médicos:

Médico d002: [(565,655,so100002),(901,960,m02),(1026,1101,so100005),(1380,1440,c02)]

Médico d003: [(720,790,m01),(910,980,m02),(1026,1101,so100005)]

Médico d005: [(565,655,so100002),(1026,1101,so100005)]

Médico d001: [(720,790,m01),(836,896,so100001),(1080,1140,c01)]

Médico n002: [(836,896,so100001)]

Médico n004: [(836,896,so100001),(836,896,so100001)]

Médico n006: [(836,896,so100001)]

Médico a002: [(836,896,so100001)]

Médico d004: [(520,655,so100002),(836,896,so100001),(791,896,so100001),(981,1101,so100005)]

Médico n003: [(520,655,so100002),(791,896,so100001),(981,1101,so100005)]

Médico a001: [(655,699,so100002),(896,940,so100001),(1101,1145,so100005)]

Médico n001: [(565,655,so100002),(836,896,so100001),(1026,1101,so100005)]

Médico n005: [(565,655,so100002),(836,896,so100001),(1026,1101,so100005)]

Tempo de geracao da solucao:0.02681279182434082

true.

Segunda Heurística

Na segunda heurística, conhecida como heurística de ocupação, seguimos a mesma lógica utilizada anteriormente. Inicialmente, a cirurgia a ser agendada é determinada com base na ocupação dos médicos. Em seguida, realiza-se a avaliação de cada membro do staff e a verificação da disponibilidade de horários. O código responsável pela inclusão de todo o staff e das fases da cirurgia permanece idêntico nesta abordagem.

Resultado gerado:

```
Todas as cirurgias foram marcadas com sucesso! Resultados de agendamento para o dia 20241028:
Sala de Operações orl
  Agenda para sala orl no dia 20241028: [(520,669,so100001),(670,849,so100002),(981,1145,so100003)]
Agendas dos Médicos:
  Médico d001: [(565,625,so100001),(720,790,m01),(1080,1140,c01)]
  Médico n002: [(565,625,so100001)]
  Médico n004: [(565,625,so100001),(565,625,so100001)]
  Médico n006: [(565,625,so100001)]
  Médico a002: [(565,625,so100001)]
  Médico d003: [(720,790,m01),(910,980,m02),(1026,1101,so100003)]
  Médico d002: [(715,805,so100002),(901,960,m02),(1380,1440,c02)]
  Médico d005: [(715,805,so100002)]
  Médico d004: [(565,625,so100001),(520,625,so100001),(670,805,so100002),(1026,1101,so100003),(981,1101,so100003)]
  Médico n003: [(520,625,so100001),(670,805,so100002),(981,1101,so100003)]
  Médico a001: [(625,669,so100001),(805,849,so100002),(1101,1145,so100003)]
  Médico n001: [(565,625,so100001),(715,805,so100002),(1026,1101,so100003)]
  Médico n005: [(565,625,so100001),(715,805,so100002),(1026,1101,so100003)]
Tempo de geracao da solucao:0.01819300651550293
true.
```

5-Conclusão

Concluindo, a escalabilidade de problemas de otimização torna inviável avaliar todas as soluções devido ao crescimento exponencial das combinações. Nesse contexto, a aplicação de heurísticas mostrou-se essencial, superando limitações práticas de hardware e software, ao gerar soluções eficientes com diferenças mínimas no tempo da última cirurgia agendada.

Em cenários reais, onde múltiplas cirurgias precisam ser organizadas envolvendo diversas salas e equipes, a escolha adequada de heurísticas é crucial para maximizar a eficiência. As heurísticas desenvolvidas demonstraram tempos de resposta consistentes, independentemente do número de cirurgias, devido às restrições diárias das salas. A inclusão de outros profissionais, como equipes de suporte, foi integrada sem impacto significativo na complexidade do sistema, exigindo apenas ajustes na avaliação de disponibilidade.

Esses resultados destacam a relevância de soluções heurísticas bem projetadas para enfrentar problemas de escalabilidade, garantindo eficiência operacional em cenários dinâmicos e complexos.